Logistics System – Backend Task

Overview

In this task, you are required to develop a **Logistics System**, where the operations team and drivers manage delivery orders and delivery runs. The system supports the core operations of logistics: creating delivery orders, building delivery runs, assigning drivers, executing deliveries, and banking collected cash. Runs go through a defined **lifecycle workflow**, and the system enforces **role-based access control** so that managers, dispatchers, and drivers can only perform authorized actions.

While there are **mandatory** requirements that must be completed, there is also a **bonus** requirement. The bonus requirement is optional and should only be implemented if you have extra time and it does not compromise the quality of the mandatory requirements. Achieving the bonus requirement is not essential but will be considered a plus.

This task is designed to evaluate your technical skills, problem-solving abilities, and attention to detail.

Technology Choice

- **Back End:** You may build the backend with Django or Frappe. Both are acceptable; **Frappe is the recommended choice and will be considered a plus**.
- **Database:** Your choice.

Delivery operations: read carefully:

- **Build Run:** Assigns a set of open orders to one driver as an ordered list of stops.
- **Start Run:** Transitions the run and its assigned orders to an active state, dispatching the driver.
- **Mark Stop:** Logs the outcome of an individual stop as either delivered or failed.
- **Complete Run:** Finalizes the driver's current run once all stops are accounted for.
- **Bank Cash:** Record the cash collected from the run and logs.

Back End

a. Data Models

Driver

- Driver Name
- Phone Number (required)
- Active (yes / no)
- Max Stops Per Run.
- Status: Available / On Run / Inactive

Order

- Customer Name (required)
- Customer Phone
- Address (required)
- Cash Amount
- Priority: High / Medium / Low
- Status: Open / Assigned / En Route / Delivered / Failed / Cash Banked
- Assigned Driver
- Delivery Run
- Created Date, Delivered Date

Delivery Run

- Driver
- Run Status: Draft / Assigned / En Route / Completed / Cash Banked / Cancelled
- Total Cash Collected
- Started Date, Completed Date
- Cash Banked Date, Cash Banked Location
- Delivery Stops (child table)

Delivery Stop (Child Table)

- Related Order
- Stop Sequence
- Customer Name
- Address
- Cash Amount
- Stop Status: Assigned / En Route / Delivered / Failed
- Delivered Date
- Failed Reason (if failed)

b. Validations & Business Logic

1. Validate all required fields are filled (e.g., Phone number for drivers, Customer Name and Address for orders).
2. Validate that a driver's maximum stops per run is greater than zero.
3. Validate that order cash amounts cannot be negative.
4. Inactive drivers cannot be assigned to new runs, and a driver already On Run cannot be assigned to another active run.
5. Only Open orders can be assigned to a new run.
6. Handle errors and return appropriate HTTP status codes and clear error messages.

c. Workflow: Run Lifecycle

Model the delivery process as a workflow with defined stages. Implement each step exposed through a secure API action:

- **Build Delivery Run:** Validate driver availability. Select Open orders up to the driver's maximum limit. Add selected orders to a new run as delivery stops. Change orders from Open to Assigned.
- **Start Delivery Run:** Validate the run has not started. Change run, orders, and stops to En Route. Change driver status to On Run.
- **Mark Stop as Delivered:** Change stop and linked order status to Delivered. Add order cash to the run's total collected.
- **Mark Stop as Failed:** Require a failed reason. Change stop and linked order status to Failed.
- **Complete Delivery Run:** Ensure all stops are Delivered or Failed. Change run to Completed. Change driver to Available.
- **Bank Cash:** Change run and delivered orders to Cash Banked.

d. Security & Permissions

Implement **role-based access control (RBAC)**:

- **Manager:** Full access to everything.
- **Dispatcher:** Create orders, manage drivers, and build/start delivery runs.
- **Driver:** View and update only the delivery stops assigned to them.

4. Bonus Requirements (Optional)

- **Dashboard:** One page showing total open orders, active drivers, runs currently en route, and total cash collected today. Include a list of active delivery runs and a quick action to build a run for a selected driver. It need not be visually perfect - clean, usable, and easy to understand is enough.

Task Submission

Submit your task via a **public GitHub repository**. Follow these steps:

1. A README with setup, installation, and migration steps.
2. A short explanation of the backend design and the status flow.

Expected Time: 3 days for a backend submission. Quality matters more than speed - the goal is clean thinking, strong backend and framework understanding, and a practical business workflow built end to end.

Task Evaluation Criteria

Submissions will be evaluated on:

1. **Backend:** Correct app structure, clean Python logic, proper validations, correct relationships, safe transaction handling, clear errors, and good status-transition logic.
2. **Data model:** Correct relationships between driver, order, run, and stop; proper child-table usage; sensible field types and naming.
3. **API:** Clear actions, proper validation and permission checks, no unsafe assumptions, good response structure, handles failure cases.
4. **Communication:** a clear README, and clearly stated assumptions.